Vector Extension Support for the Kites RISC-V Simulator

Scope Document

RISC-V "V" Standard Extension for Vector Operations, version 1.0 (RVV 1.0)

Document	Scope document
Version	1.2
Author	Raam M
Repository	`https://github.com/Raam-M/Kites` (fork of `https://github.com/hghit-harshit/Kites`)
Branch	`vector-extension`, baseline commit `7ddb6f3`
Status	For review. The scope is frozen once the open questions in Section 12 are answered.

Purpose of this document

This document defines *what* will be added to Kites to support the RISC-V Vector extension, separates the *compulsory* work from work that goes *over and above* it, and states *how* the work will be designed, validated and documented.

Revision history

Version	Change
1.0	First complete draft: baseline analysis, scope tiers, requirements, approach, validation and documentation plans.
1.1	Removed schedule and effort estimates; technical rationale only.
1.2	Aligned with the implementation: D9 revised, module table and integration notes updated, findings F16–F19 added.

Contents

1 Executive summary **3**

2 Baseline: Kites today **4**

2.1 Architecture overview 4

2.2 Feature baseline 4

2.3 Findings that shape the plan 5

3 RVV 1.0 in brief **6**

3.1 Programmer's model 6

3.2 Element categories and register groups 7

3.3 Instruction encoding 8

3.4 Extension subsets 8

3.5 What vector code looks like 9

4 Goals, non-goals and design decisions **9**

4.1 Goals 9

4.2 Non-goals 10

4.3 Design decisions 10

5 Scope **11**

5.1 Tier A1 — compulsory core 12

5.2 Tier A2 — compulsory spec completion 13

5.3 Tier B — over and above (in priority order) 13

5.4 Tier C — out of scope 14

6 Requirements and acceptance criteria **14**

7 Technical approach **17**

7.1 Module structure 17

7.2 Execution flow 17

7.3 The shared element loop 18

7.4 Integration with existing subsystems 18

8 Implementation plan **18**

8.1 Phases 18

8.2 Ways of working 19

9 Validation and verification plan **20**

9.1 Principles . . . 20
9.2 Test levels . . . 20
9.3 Configuration test matrix . . . 21
9.4 Reference oracles and golden sources . . . 21
9.5 Example programs . . . 22
9.6 Quality gates . . . 23
9.7 Traceability and defect tracking . . . 23

10 Documentation plan **23**

11 Risks and mitigations **24**

12 Open questions **24**

A Instruction inventory **26**

B Assembly syntax forms **27**

C Impact on existing Kites files **28**

D Glossary **29**

1 Executive summary

Kites is a RISC-V (RV64) assembler, simulator and IDE. Today it supports the I, M, F and D extensions plus CSR instructions on a single-cycle processor and on four variants of a classic 5-stage pipeline. This project adds the RISC-V Vector extension, version 1.0 ("RVV 1.0", the standard **V** extension [1]). With the extension in place:

- the assembler, disassembler and editor understand the vector instruction set;
- the simulator has a vector register file, the seven vector CSRs and a vector execution unit;
- stepping, undo/redo, breakpoints, register dumps, the CLI, the profiler and the GUI register view are all vector-aware;
- every supported instruction is validated against independent references, and the work is documented for both users and developers.

Scope at a glance

Compulsory — A1 Core. Vector state and CSRs; `vsetvli/vsetivli/vsetvl`; all non-segment loads and stores; integer arithmetic; masking and tail/mask policies; integer reductions; mask instructions; permutations; legality checks. Full toolchain integration (assembler, disassembler, editor), execution on the single-cycle processor, undo/redo, register dumps, a GUI vector-register view, the CLI, tests and documentation.

Compulsory — A2 Spec completion. Fixed-point arithmetic, floating-point vector instructions and reductions, segment and fault-only-first loads/stores, and the standard pseudo-instructions. Together with A1, this is everything needed to claim conformance with the **V** extension.

Over and above — B. Vector execution in the 5-stage pipelines with vector hazard detection; a vector timing model (lanes, chaining); a rich element-wise vector viewer; configurable VLEN; vector profiling; large-scale randomised differential testing against Spike; a vector unit in the circuit view; further extensions (Zvfh, Zvbb, ...).

Out of scope — C. Privileged vector state (`mstatus.VS`), trap-based resumption, vector cryptography, compiling C to vector code.

Table 1: Key parameters of the planned implementation.

Specification target	RVV 1.0 as published in the Unprivileged ISA manual, release 20260120, Chapter 30 [1]
Vector register length	VLEN = 128 bits by default (minimum for **V**); a single parameter, power of two
Maximum element width	ELEN = 64 bits; SEW $\in \{8, 16, 32, 64\}$, LMUL $\in \{\frac{1}{8}, \frac{1}{4}, \frac{1}{2}, 1, 2, 4, 8\}$
Architectural state	32 vector registers `v0`–`v31`; 7 CSRs (`vstart`, `vxsat`, `vxrm`, `vcsr`, `vl`, `vtype`, `vlenb`)
Instruction count	627 assembly mnemonics in **V** (375 excluding the 252 segment load/store variants): 238 in A1, 389 in A2 (Appendix A)
Primary execution model	Single-cycle processor (RVSS); vector instructions execute atomically
Pipelined models	Clean rejection in A1; full integration is stretch goal B1

Deliverables. Source code on the `vector-extension` branch; a GoogleTest suite and example programs; three LaTeX/PDF documents (this scope document, a design document and a validation report); Markdown user documentation (`docs/VECTOR.md`, plus updates to the README, `docs/CLI.md` and `docs/TESTING.md`). Section 10 has the full list.

2 Baseline: Kites today

2.1 Architecture overview

Figure 1 shows the parts of Kites that matter for this project. Components this project adds are shaded; existing components that need changes are outlined in orange.

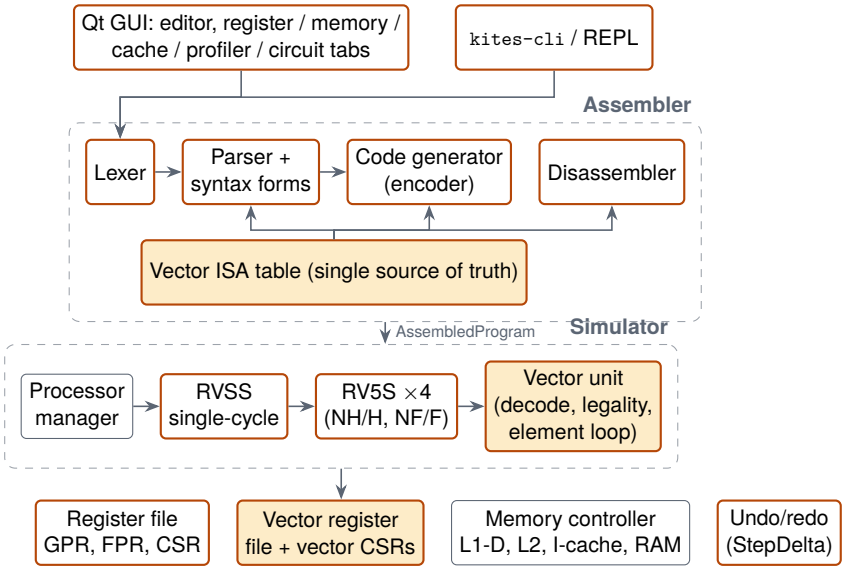


Figure 1: Kites components relevant to the vector extension. Shaded boxes are new, orange-outlined boxes are modified. The vector unit is shared by all processor models; the vector ISA table drives the parser, encoder, disassembler and executor.

2.2 Feature baseline

Table 2: What Kites supports at the baseline commit.

Area	Baseline
ISA	RV64I, M, F, D; CSR instructions (`csrrw`, ...); a large set of pseudo-instructions, plus user-defined pseudo-instructions
Processor models	`RVSS` single-cycle; four 5-stage pipelines `RV5S` {no hazard detection, hazard detection} × {no forwarding, forwarding}
Memory system	64-bit byte-addressed sparse main memory; L1 data cache, L2 cache, instruction cache with configurable replacement policies (including Lua-scripted custom policies)
Tools	Qt 6.9 GUI (editor with highlighting and autocompletion, register/memory/cache/profiler/circuit tabs, breakpoints, undo/redo), headless `kites-cli` and REPL
Testing	GoogleTest suite (about 20 files) run through CTest, with an optional gcovr coverage target

2.3 Findings that shape the plan

A review of the code base at commit `7ddb6f3` found points F1–F15; F16–F19 were found during implementation. Each one is an integration risk or a design constraint, addressed in Section 7.

Table 3: Findings from the code review (F1–F15, at commit `7ddb6f3`) and from the implementation (F16–F19).

ID	Finding	Implication for the vector work
F1	`RegisterFile` stores GPRs and FPRs as `uint64_t` arrays and CSRs as a flat 4096-entry array. A `RegisterType::VECTOR` enum value exists but is unused. Only `fflags`, `frm` and `fcsr` have assembler names.	A VLEN-wide vector register file is needed. Vector CSRs must be named, `vl`/`vtype`/`vlenb` must be read-only, and `vcsr` must be a composite view of `vxrm` and `vxsat`.
F2	`TokenType::VEC_REGISTER` is declared but the lexer never produces it.	Placeholder to build on; no existing behaviour to preserve.
F3	The lexer treats "`.`" as an identifier character, and an unknown identifier after an opcode or comma becomes a `LABEL_REF`.	`vadd.vv` already lexes as one opcode (good), but `v0.t` lexes as a single identifier, and `e32`, `m1`, `ta`, `ma` would become label references. Lexing has to be context-sensitive.
F4	The parser has one hand-written `parse_*` function per syntax form (27 forms, 177 entries in the syntax map).	RVV needs about 30 new operand shapes. Writing them in the same style would add thousands of near-duplicate lines, so vector syntax will use a generic operand-pattern matcher instead.
F5	`ICUnit` has no mask (`vm`) field and no vtype field. The opcode buffer holds 20 characters.	Add the fields. The longest RVV mnemonic, `vfwcvt.rtz.xu.f.v`, has 17 characters, so it fits.
F6	Vector loads/stores share the LOAD-FP (`0000111`) and STORE-FP (`0100111`) opcodes with `flw`/`fld`/`fsw`/`fsd`; only the `width` field tells them apart. `isFInstruction`/`isDInstruction` already ignore vector widths, but the RVSS control unit sets FP load/store signals from the opcode alone.	Without explicit routing, vector memory instructions would silently mis-execute as scalar loads/stores. Decoding must check `width` before taking the F/D path. Existing F/D tests guard against regressions.
F7	Undo/redo records `RegisterChange{index, type, old, new}` with 64-bit values. `MemoryChange` already stores byte vectors.	A 128-bit or wider register does not fit. Add a byte-vector `VectorRegisterChange` and a matching undo/redo case. Vector stores can reuse `MemoryChange`.
F8	Scalar FP execution writes the raw exception flags of each instruction into `fcsr` (0x003). This overwrites the accrued flags and the `frm` bits and does not touch `fflags` (0x001).	Vector FP accrues its flags into `fflags` itself (D9); the scalar issue is reported separately.
F9	The 64-bit high multiplies use `__int128`, with a TODO noting that MSVC lacks it.	`vmulh*`, `vwmul*` and `vsmul` at SEW=64 need a portable 64×64→128 helper.
F10	In the pipelines, decode reads GPR or FPR operands and the hazard unit compares the 5-bit `rs1`/`rs2`/`rd` fields as GPR/FPR numbers.	Vector instructions reuse these fields for vector registers, which would cause false hazards and wrong operands. They must be guarded in A1 and integrated properly in B1.
F11	`DumpRegisters` writes CSRs (taken from `csr_to_address`), GPRs and FPRs to `registers_dump.json`, which both the CLI and the GUI read.	Named vector CSRs will appear in the dump automatically. Vector registers need a new JSON section.

Table 3: Findings (continued).

ID	Finding	Implication for the vector work
F12	The profiler groups instructions by `InstructionType` (R/I/S/B/U/J/F).	Add a `V_TYPE` category so vector instructions are not counted as unknown.
F13	`resources/instructions/instructions.json` drives syntax highlighting and autocompletion.	Add a `VInstruction` list, ideally generated from the ISA table.
F14	There is no trap model. Runtime errors are C++ exceptions, surfaced through `runErrorSignal` with the PC mapped back to a source line.	Illegal vector usage reuses this path and stops the run with a precise message (decision D5).
F15	The README says the pipelines support only I and M, but pipelined F/D support and tests exist.	Documentation drift; fix it while updating the README.
F16	The scalar FP ALU deviates from the spec: $x/0$ returns NaN instead of $\pm\infty$, the exception-flag bits are in reverse order, a NaN converted to an integer does not give the maximum value, and RMM is not supported.	Vector FP cannot reuse these routines without inheriting the deviations (D9). The scalar issues are reported separately.
F17	A data label named `data`, `text` or `bss` makes the assembler loop forever or reject the program.	Example programs avoid these names; reported separately.
F18	Single-precision values are stored zero-extended in the FPRs instead of NaN-boxed.	`vfmv.f.s` follows the same convention, and `.vf` operands use the low 32 bits.
F19	`test/test_main.cpp` includes `windows.h` unconditionally, so the tests do not build on macOS or Linux.	Fixed with an `#ifdef _WIN32` guard.

3 RVV 1.0 in brief

This section summarises the parts of the specification [1] that drive the plan. It is a primer for readers new to RVV, not a replacement for the specification.

3.1 Programmer's model

RVV adds 32 vector registers `v0`–`v31`, each VLEN bits wide. Software does not hard-code VLEN. It requests an *application vector length* (AVL) and a vector type with `vsetvli`, and the hardware returns the number of elements `vl` it will process. A loop that processes `vl` elements per iteration until AVL reaches zero is called *strip-mining*. The same binary therefore runs on any VLEN.

- **SEW** (selected element width) is 8, 16, 32 or 64 bits.
- **LMUL** (length multiplier) groups 2, 4 or 8 consecutive registers into one operand, or uses a fraction ($\frac{1}{2}, \frac{1}{4}, \frac{1}{8}$) of a register.
- VLMAX = LMUL × VLEN/SEW, and `vl` ≤ VLMAX. Kites will set `vl` = min(AVL, VLMAX) (decision D2).
- **Masking**: most instructions accept an optional `v0.t` operand. Element i is active when bit i of `v0` is 1.
- **Policies**: tail elements ($i \geq$ `vl`) and masked-off elements are either *undisturbed* (they keep

their old value) or *agnostic* (old value or all ones), as selected by `vta` and `vma`.

- **vstart** gives the first element to process, so an interrupted instruction can resume. Every vector instruction resets it to 0.

Table 4: Vector CSRs (URW = user read/write, URO = user read-only).

CSR	Address	Access	Purpose
`vstart`	0x008	URW	Index of the first element to execute
`vxsat`	0x009	URW	Fixed-point saturation flag
`vxrm`	0x00A	URW	Fixed-point rounding mode (rnu, rne, rdn, rod)
`vcsr`	0x00F	URW	Composite: `vxrm` in bits [2:1], `vxsat` in bit [0]
`vl`	0xC20	URO	Current vector length
`vtype`	0xC21	URO	Current vector type: `vill`, `vma`, `vta`, `vsew`, `vlmul`
`vlenb`	0xC22	URO	VLEN/8, the register width in bytes

The `vtype` register. Bit XLEN−1 is `vill`; bits [XLEN−2:8] are reserved; bit 7 is `vma`; bit 6 is `vta`; bits [5:3] are `vsew`; bits [2:0] are `vlmul`. `vsew` = 000/001/010/011 selects SEW = 8/16/32/64. `vlmul` = 101/110/111/000/001/010/011 selects LMUL = $\frac{1}{8}/\frac{1}{4}/\frac{1}{2}/1/2/4/8$, and 100 is reserved. An unsupported setting sets `vill`, and then every vector instruction except `vset{i}vl{i}` is illegal.

Table 5: VLMAX for VLEN = 128, ELEN = 64. A dash marks combinations with SEW > LMUL×ELEN, which set `vill`.

SEW \ LMUL	$\frac{1}{8}$	$\frac{1}{4}$	$\frac{1}{2}$	1	2	4	8
8	2	4	8	16	32	64	128
16	—	2	4	8	16	32	64
32	—	—	2	4	8	16	32
64	—	—	—	2	4	8	16

3.2 Element categories and register groups

Figure 2 shows how an instruction treats each element. Getting these categories right for every instruction family is the main source of subtle bugs, so the design (Section 7.3) handles them in one shared element loop.

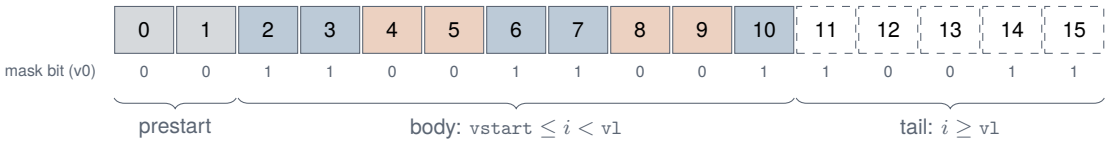


Figure 2: Element categories for a masked instruction with VLMAX = 16, `vstart` = 2 and `vl` = 11. Grey: prestart elements, never written. Blue: active body elements (mask bit 1), computed normally. Orange: inactive body elements (mask bit 0), which follow `vma`. Dashed: tail elements, which follow `vta`.

With LMUL > 1, an operand is a group of consecutive registers whose first register number must be a multiple of LMUL. Element i of a group that starts at vn lives in register $n + \lfloor i/(\text{VLEN}/\text{SEW}) \rfloor$. Within a register, element 0 occupies the least significant bits. Figure 3 shows an example. Mask values always use a single register, one bit per element.

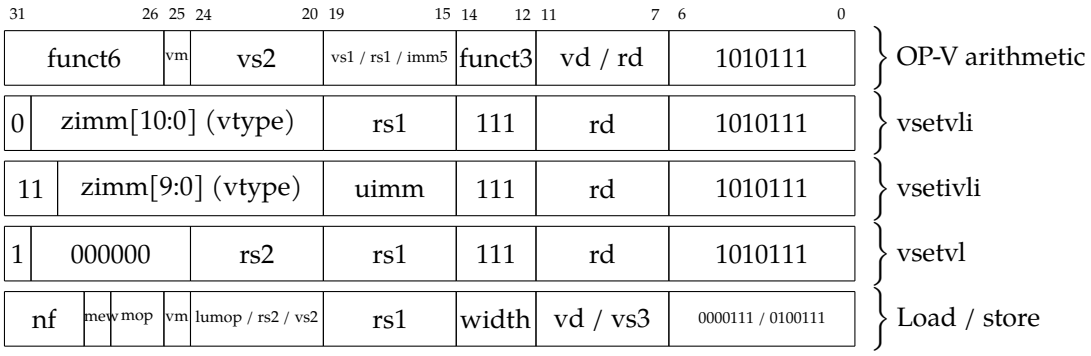


Figure 3: Register group `v2`–`v3` with VLEN = 128, SEW = 32 and LMUL = 2 (VLMAX = 8). An instruction naming `v3` as a group under LMUL = 2 is illegal.

Effective widths. Loads and stores encode their own element width EEW. The register group then uses EMUL = (EEW/SEW) × LMUL, which must lie in $[\frac{1}{8}, 8]$. Widening instructions write 2·SEW elements into 2·LMUL registers, and narrowing instructions read them. The specification restricts how source and destination groups of different widths may overlap, and how a masked destination may overlap `v0`. These rules are implemented as explicit, individually tested legality checks (requirement group VL).

3.3 Instruction encoding

Arithmetic and configuration instructions use the new major opcode OP-V (`1010111`). `funct3` selects the operand category: OPIVV `000`, OPFVV `001`, OPMVV `010`, OPIVI `011`, OPIVX `100`, OPFVF `101`, OPMVX `110`, OPCFG `111`. Loads and stores reuse LOAD-FP and STORE-FP and are told apart from scalar FP accesses by the `width` field: `000`, `101`, `110`, `111` mean EEW = 8, 16, 32, 64.



For loads and stores, `mop` selects the addressing mode: `00` unit-stride, `01` indexed-unordered, `10` strided, `11` indexed-ordered. For unit-stride accesses, `lumop`/`sumop` selects a variant: `00000` plain, `01000` whole register, `01011` mask, `10000` fault-only-first (loads only). `nf` encodes the number of segment fields minus one, and `mew` must be 0.

3.4 Extension subsets

The specification defines embedded subsets Zve32x, Zve32f, Zve64x, Zve64f and Zve64d (Table 6). The full **V** extension depends on Zvl128b (VLEN ≥ 128) and Zve64d, and on the scalar F and D extensions, all of which Kites already has. It includes every vector instruction listed in Appendix A. Half-precision vector FP (Zvfh), vector bit manipulation (Zvbb) and vector cryptography are separate extensions and are not part of **V**.

Table 6: Standard vector subsets [1].

Extension	Minimum VLEN	Supported EEW	FP32	FP64
Zve32x	32	8, 16, 32	no	no
Zve32f	32	8, 16, 32	yes	no
Zve64x	64	8, 16, 32, 64	no	no
Zve64f	64	8, 16, 32, 64	yes	no
Zve64d	64	8, 16, 32, 64	yes	yes
V	128	8, 16, 32, 64	yes	yes

3.5 What vector code looks like

The strip-mined loop below adds two `int32` arrays. It is the first end-to-end test program.

```
# void vvadd(size_t n, const int32_t *a, const int32_t *b, int32_t *c)
# a0 = n, a1 = a, a2 = b, a3 = c
vvadd:
    vsetvli t0, a0, e32, m1, ta, ma   # t0 = vl = min(n, VLMAX)
    vle32.v v0, (a1)                  # load a[0 .. vl-1]
    sub     a0, a0, t0                # n -= vl
    slli    t0, t0, 2                 # bytes processed = vl * 4
    add     a1, a1, t0
    vle32.v v1, (a2)                  # load b[0 .. vl-1]
    add     a2, a2, t0
    vadd.vv v2, v0, v1                # c = a + b, element-wise
    vse32.v v2, (a3)
    add     a3, a3, t0
    bnez    a0, vvadd                 # loop until all n elements are done
    ret
```

4 Goals, non-goals and design decisions

4.1 Goals

G1 **Correctness.** Every supported instruction gives architecturally correct results for every legal combination of SEW, LMUL, `vl`, `vstart`, mask and policy, matching an independent reference.

G2 **Integration.** Vector code works in every Kites workflow: editing, assembling, running, stepping, undo/redo, breakpoints, register and memory inspection, CLI dumps and profiling.

G3 **Determinism.** Identical programs give identical results. Every implementation-defined choice in the spec is fixed and documented (Section 4.3).

G4 **Robustness.** Illegal or reserved usage is rejected with a precise message that points to the source line. Nothing is silently mis-executed.

G5 **No regressions.** Scalar behaviour, scalar performance and all existing tests are unaffected.

G6 **Maintainability.** One table-driven definition of the vector ISA is shared by the assembler, disassembler and executor, and the code follows `docs/coding_convention.md`.

4.2 Non-goals

- A cycle-accurate model of any particular commercial vector processor.
- Compiler support or auto-vectorisation. Kites assembles hand-written assembly.
- Supervisor and machine-level features (see Tier C, Section 5.4).

4.3 Design decisions

Table 7: Design decisions.

ID	Decision	Rationale
D1	VLEN = 128 and ELEN = 64 by default. VLEN is one configuration value (a power of two from 128 to 1024).	128 is the minimum for **V** (Zvl128b). With VLEN a parameter from the start, B4 needs only UI work.
D2	`vl` = min(AVL, VLMAX).	Satisfies all three spec constraints on `vl` and matches Spike.
D3	Agnostic elements are left *undisturbed* by default. An "all ones" fill can be selected (B4). Mask-destination tails are also left undisturbed.	Both behaviours are legal. Undisturbed is deterministic; the all-ones mode exposes code that depends on agnostic values.
D4	An unsupported vtype (reserved `vsew/vlmul`, SEW > LMUL×ELEN, or non-zero reserved bits) sets `vill`=1, `vl`=0 and the other `vtype` bits to 0. The reserved form `vsetvli x0, x0` that would change VLMAX also sets `vill`. Reset state: `vill`=1, `vl`=0.	This is the behaviour the spec prescribes or permits, and it is what the spec recommends for reset.
D5	Illegal-instruction conditions stop the run with a runtime error that names the violated rule. All checks run *before* any state changes, and results are staged and committed together, so state is unchanged after an error.	Kites has no trap handler (F14). Atomic commit keeps undo/redo and error reporting consistent.
D6	`vstart` is honoured (elements below it are untouched) and reset to 0 after every vector instruction. A `vstart` above the largest element index is rejected.	Spec behaviour. Without traps, a non-zero `vstart` only arises from an explicit CSR write.
D7	Each vector instruction executes atomically in one simulator step. In RVSS it retires as one instruction in one cycle.	Matches the single-cycle abstraction. Realistic timing is stretch goal B2.
D8	Element memory accesses go through `MemoryController` one element at a time, in ascending element order. Misaligned element addresses are allowed.	Caches see vector traffic. Ascending order satisfies the indexed-ordered semantics and is a valid unordered order. Kites performs no alignment checks for scalar accesses, so allowing misaligned elements is consistent.

Table 7: Design decisions (continued).

ID	Decision	Rationale
D9	Vector FP uses its own element routines: host IEEE arithmetic with the rounding mode from `frm`, flags accrued into `fflags` in the RISC-V bit layout, and the RISC-V rules applied explicitly (canonical NaN, minimumNumber/maximumNumber, saturating float-to-integer conversion, round-to-odd, bit-exact 7-bit estimates).	The scalar FP routines deviate from the spec (F16). RMM rounds ties to even in arithmetic because the host has no such mode; conversions to integer honour RMM.
D10	One vector ISA table (mnemonic, encoding, operand pattern, semantic class, flags) drives the encoder, parser, disassembler, executor dispatch, `instructions.json` and the instruction list in the documentation. It is cross-checked automatically against `riscv-opcodes` [4].	Avoids the duplication noted in F4 and keeps the assembler, disassembler and executor consistent by construction.
D11	Assembly syntax follows GNU `as` and LLVM: spec operand order, a "`, v0.t`" mask suffix, vtype written as "`e32, m2, ta, ma`", and `tu, mu` when the policy is omitted.	Source compatibility with standard toolchains; LLVM can produce golden encodings.
D12	Until B1 is done, the 5-stage models stop with "vector instructions are supported only on the single-cycle processor" when they decode a vector instruction.	Guarantees no silent mis-execution (F10) without blocking A1.
D13	New code lives in `src/processor/vector/` and `src/assembler/parse_formats/v_formats.cpp`. Existing files receive only small, well-defined hooks.	Limits regression risk and keeps the change reviewable.
D14	Mnemonics that are not yet implemented are rejected by the assembler with "not supported".	Partial support never mis-executes.

5 Scope

The work is split into four tiers:

- **A1 — Compulsory core**: the minimum for Kites to support RVV integer programs correctly, end to end.
- **A2 — Compulsory spec completion**: the rest of the **V** extension. A1 + A2 is what "Kites supports the V extension" means.
- **B — Over and above**: extensions beyond conformance, in priority order.
- **C — Out of scope**: explicitly not done.

Instruction counts refer to Appendix A.

5.1 Tier A1 — compulsory core

Table 8: Tier A1 items: the compulsory core.

ID	Item	Contents
A1.1	Vector architectural state	32 × VLEN register file with element and register-group accessors. CSRs `vstart`, `vxsat`, `vxrm`, `vcsr` (composite view), `vl`, `vtype`, `vlenb`, with URW/URO enforcement and assembler names. Reset state as in D4.
A1.2	Configuration instructions	`vsetvli`, `vsetivli`, `vsetvl`. AVL rules, including the special `rs1=x0` cases. `vill` handling (3 mnemonics).
A1.3	Loads and stores (non-segment)	Unit-stride, strided (including zero and negative strides), indexed-ordered and indexed-unordered, mask (`vlm.v`/`vsm.v`) and whole-register forms, for EEW 8–64, with EEW/EMUL legality checks (54 mnemonics).
A1.4	Integer arithmetic	Add/sub/reverse-sub; widening add/sub (including `.w` forms); zero/sign extension; add-with-carry and subtract-with-borrow; bitwise; shifts; narrowing shifts; compares; min/max; multiply and high multiply; divide/remainder (RISC-V divide-by-zero and overflow results); widening multiply; single-width and widening multiply-add; merge; `vmv.v.*` (139 mnemonics).
A1.5	Masking and policies	`v0.t` masking for every maskable instruction; `vta`/`vma`; prestart, body and tail handling (Figure 2); mask register layout.
A1.6	Integer reductions	Single-width and widening reductions (10 mnemonics).
A1.7	Mask instructions	Eight mask-logical operations, `vcpop.m`, `vfirst.m`, `vmsbf/vmsif/vmsof.m`, `viota.m`, `vid.v` (15 mnemonics).
A1.8	Integer permutations	`vmv.x.s`, `vmv.s.x`, slides (up, down, 1up, 1down), `vrgather` (`.vv`/`.vx`/`.vi`), `vrgatherei16.vv`, `vcompress.vm`, `vmv<nr>r.v` (17 mnemonics).
A1.9	Legality and errors	Register-group alignment; overlap rules (widening, narrowing, mask, `v0`); EMUL range; reserved encodings (`mew`=1, reserved widths and `funct6` values); `vill`; `vstart` range; writes to read-only CSRs. Each is reported as a runtime error with a source line (D5).
A1.10	Assembler and editor	Lexer tokens (`v0`–`v31`, `v0.t`, vtype fields); operand patterns; immediate range checks (simm5, uimm5, `vsetivli` uimm and zimm); encoder; disassembler; error messages that show the expected syntax; `instructions.json` entries for highlighting and autocompletion; `csrr`/`csrw`/`csrs`/`csrc` pseudo-instructions so programs can read `vl`, `vtype` and `vlenb`.
A1.11	Single-cycle integration	Decode routing (OP-V, plus vector LOAD-FP/STORE-FP selected by `width`); execution through the vector unit; atomic commit; `vstart` reset; instruction and cycle counting; undo/redo of vector register, CSR and memory changes; reset; breakpoints and stepping inside vector loops.
A1.12	Pipelined models	Clean rejection of vector instructions (D12), with tests on all four variants.
A1.13	Observability	`registers_dump.json` with vector registers and vector CSRs; CLI documentation; a *Vector* section in the GUI register view (`v0`–`v31` in hex, plus vector CSRs, with change highlighting); a `V-Type` category in the profiler.
A1.14	Validation	Everything in Section 9 that applies to A1 items, including the full regression suite.

Table 8: Tier A1 items: the compulsory core (continued).

ID	Item	Contents
A1.15	Documentation	Everything in Section 10 that applies to A1 items.

5.2 Tier A2 — compulsory spec completion

Table 9: Tier A2 items: compulsory spec completion.

ID	Item	Contents
A2.1	Fixed-point arithmetic	Saturating add/sub; averaging add/sub; fractional multiply `vsmul`; scaling shifts; narrowing clips. `vxrm` rounding modes rnu, rne, rdn, rod, and `vxsat` (32 mnemonics).
A2.2	Floating-point arithmetic	For SEW = 32 (F) and SEW = 64 (D): add/sub/reverse-sub; widening add/sub; mul/div/reverse-div; widening mul; 8 single-width and 4 widening fused multiply-adds; sqrt; `vfrsqrt7`/`vfrec7` (bit-exact spec tables); min/max; sign injection; compares; classify; merge; `vfmv.v.f`; single-width, widening and narrowing conversions, including `rtz` and `rod` forms (91 mnemonics).
A2.3	FP reductions and permutations	Ordered and unordered sum, min and max reductions, plus their widening forms (6); `vfmv.f.s`, `vfmv.s.f`, `vfslide1up/down.vf` (4).
A2.4	Segment and fault-only-first	Segment loads and stores with NFIELDS 2–8 in every addressing mode (252 mnemonics). Fault-only-first loads `vle<eew>ff.v` (4), which trim `vl` when a later element would fault (in Kites, an out-of-range address).
A2.5	Standard pseudo-instructions	`vneg.v`, `vnot.v`, `vwcvt(u).x.x.v`, `vncvt.x.x.w`, `vmsgt(u).vv`, `vmsge(u).vv`, `vmslt(u).vi`, `vmsge(u).vi`, the multi-instruction `vmsge(u).vx` expansions, `vfneg.v`, `vfabs.v`, `vmfgt/vmfge.vv`, `vmmv/vmclr/vmset/vmnot.m`, `vl<nr>r.v`.
A2.6	Validation and documentation	As for A1, for every A2 item.

5.3 Tier B — over and above (in priority order)

Table 10: Tier B items, in priority order.

ID	Item	Contents	Needs
B1	Pipelined vector execution	Vector instructions flow through IF/ID/EX/MEM/WB in all four RV5S models. Group-aware hazard detection on vector registers and on `vl`/`vtype`; stalls in the hazard-detecting models; forwarding of vector results in the forwarding models; scalar results (`vmv.x.s`, `vcpop.m`, `vsetvl rd`) through the existing paths; pipeline view labels.	A1
B2	Vector timing model	Configurable number of lanes L. An instruction occupies EX for $\lceil \texttt{vl}/L \rceil$ cycles (chime model), with optional chaining. Memory cost per cache line touched. Reports cycles, CPI and speed-up over equivalent scalar code.	B1

Table 10: Tier B items (continued).

ID	Item	Contents	Needs
B3	Rich vector register viewer	Per-element view with an SEW selector, signed/unsigned/hex/float formats, LMUL group outlines, `v0` as mask bits, changed-element highlighting, a `vtype` decoder, and in-place editing.	A1.13
B4	Configurability	VLEN (128–1024) and agnostic fill policy in the Settings dialog and as CLI flags (`--vlen`, `--vector-agnostic`), persisted in the configuration.	D1
B5	Vector profiling	Vector instruction mix; elements processed; average `vl` and active-element ratio; bytes moved by unit-stride, strided and indexed accesses; cache hit rate per access pattern.	A1
B6	Randomised differential testing	Constrained random generator of vector programs; run on Spike and Kites and compare architectural state; CI job; import of `riscv-vector-tests` [8] cases.	A1
B7	Vector unit in the circuit view	Single-cycle diagram extended with the vector register file, vector ALU, mask unit and vector load/store unit, with per-instruction wire highlighting.	A1
B8	More extensions	Zvfh (half-precision vector FP; needs scalar Zfh/Zfhmin), Zvbb/Zvkb, Zvbc; selectable Zve32x/Zve64x/Zve64d subset modes for embedded profiles.	A2
B9	CLI/REPL vector commands	Print a vector register as typed elements, set a vector register, show the decoded `vtype`; element-format options for dumps.	A1
B10	Custom vector pseudo-instructions	Vector support in the custom pseudo-instruction manager, so users can define vector macros.	A2.5

5.4 Tier C — out of scope

- Privileged-architecture vector state: the `mstatus.VS` field, dirty tracking and context-switch support.
- Trap-based resumption beyond D6, since Kites has no trap handler.
- Vector cryptography (Zvk*) and other non-**V** vector extensions, except those listed in B8.
- Loading externally compiled ELF binaries, and compiling C to vector code.
- Cycle-accurate models of specific commercial cores; multi-hart memory-ordering effects of vector accesses.
- Misaligned-address exceptions (see D8) and virtual memory.

6 Requirements and acceptance criteria

Each requirement has an ID, a tier and a verification method: **T** = automated test, **R** = review/inspection, **D** = demonstration. Test names carry requirement IDs, so the validation report can trace every requirement to its tests (Section 9.7).

Table 11: Requirements, grouped by area.

ID	Requirement	Tier	Verif.
State and CSRs (VS)			

Table 11: Requirements, grouped by area (continued).

ID	Requirement	Tier	Verif.
VS-01	Kites shall provide 32 vector registers of VLEN bits, with VLEN set by one configuration value (default 128).	A1	T
VS-02	The seven vector CSRs shall exist at their spec addresses with URW/URO access. Writing `vl`, `vtype` or `vlenb` shall be an illegal instruction.	A1	T
VS-03	`vcsr` shall read as `(vxrm` $\ll$ `1) | vxsat`, and writing it shall update both fields.	A1	T
VS-04	After reset: `vtype.vill`=1, `vl`=0, `vstart`=0, `vxrm`=0, `vxsat`=0, and all vector registers zero.	A1	T
Configuration (VC)			
VC-01	`vsetvli`, `vsetivli` and `vsetvl` shall compute `vl` = min(AVL, VLMAX) and write `vl` to `rd`.	A1	T
VC-02	`rs1`=`x0`, `rd`$\neq$`x0` shall request VLMAX. `rs1`=`rd`=`x0` shall keep `vl`; if VLMAX would change, it shall set `vill` (D4).	A1	T
VC-03	Unsupported vtype values shall set `vill`=1 and `vl`=0 (D4).	A1	T
Memory (VM)			
VM-01	Unit-stride, strided, indexed (ordered and unordered), mask and whole-register loads and stores shall access exactly the bytes defined by the spec for every EEW, EMUL, `vl`, `vstart` and mask.	A1	T
VM-02	Strided accesses shall support zero, negative and non-element-multiple strides.	A1	T
VM-03	Mask loads and stores shall use EEW = 8 and `evl` = $\lceil$`vl`$/8\rceil$. Whole-register forms shall ignore `vtype` and `vl`.	A1	T
VM-04	Every vector memory access shall go through `MemoryController` (cache statistics reflect it), and stores shall be recorded for undo.	A1	T
VM-05	Segment loads and stores (NFIELDS 2–8) and fault-only-first loads shall follow the spec, including `vl` trimming.	A2	T
Arithmetic and data movement (VA)			
VA-01	Every A1 integer, reduction, mask and permutation instruction shall match the reference model bit-exactly for every legal SEW/LMUL.	A1	T
VA-02	Division by zero and signed overflow shall give the RISC-V defined results (quotient all ones or -1, remainder = dividend; overflow quotient = dividend, remainder 0).	A1	T
VA-03	Fixed-point instructions shall implement all four `vxrm` modes and set `vxsat` exactly when saturation occurs.	A2	T
VA-04	FP instructions shall honour dynamic `frm`, produce canonical NaNs, accrue `fflags`, and match the reference bit-exactly. The exception is `vfredusum`/`vfwredusum`, where any summation order is legal (Section 9.4).	A2	T
VA-05	`vfrec7.v` and `vfrsqrt7.v` shall reproduce the spec lookup tables exactly, including special cases.	A2	T
Masking and policies (VP)			
VP-01	Inactive body elements shall follow `vma`, and tail elements shall follow `vta`, using the configured agnostic fill (D3).	A1	T
VP-02	Prestart elements ($i <$ `vstart`) shall never be modified, and `vstart` shall be 0 after every vector instruction.	A1	T
Legality (VL)			

Table 11: Requirements, grouped by area (continued).

ID	Requirement	Tier	Verif.
VL-01	Every rule in A1.9 shall stop execution with a message that names the rule, the instruction and the source line, and shall leave architectural state unchanged.	A1	T
VL-02	Executing any vector instruction except `vset*` while `vill`=1 shall be illegal.	A1	T
Assembler and editor (AS)			
AS-01	Every in-scope mnemonic, in every operand form (masked and unmasked, all immediates), shall assemble to the same 32-bit word as LLVM `llvm-mc` [7].	A1/A2	T
AS-02	The disassembler shall print canonical syntax, and assembling its output shall reproduce the same word (round trip).	A1/A2	T
AS-03	Syntax errors, unknown registers and out-of-range immediates shall give an error with the line, column and expected syntax.	A1	T
AS-04	Labels whose names look like vtype fields (for example `ta:` or `e32:`) shall still work.	A1	T
AS-05	Vector mnemonics and registers shall be syntax-highlighted and offered by autocompletion.	A1	D
AS-06	The standard vector pseudo-instructions shall expand as specified.	A2	T
Simulator integration (SI)			
SI-01	On RVSS, a vector instruction shall retire as one instruction in one cycle (D7).	A1	T
SI-02	Undo after any vector instruction shall restore registers, vector registers, CSRs and memory bit-exactly, and redo shall reapply them.	A1	T
SI-03	Scalar F/D loads and stores shall be unaffected by the shared opcodes (F6).	A1	T
SI-04	On the 5-stage models, a vector instruction shall stop the run with the D12 message (A1). With B1, it shall execute correctly.	A1 / B1	T
Observability (OB)			
OB-01	`registers_dump.json` shall contain the vector CSRs and a `vector_registers` section, with a documented byte-order convention.	A1	T
OB-02	The GUI register view shall show `v0`–`v31` and the vector CSRs, and shall update on step, run, undo, redo and reset.	A1	D
OB-03	The profiler shall count vector instructions as `V-Type`.	A1	T
Non-functional (NF)			
NF-01	All existing tests shall pass, and run time on existing scalar examples shall stay within ±5% of the baseline.	A1	T
NF-02	The code shall build and pass tests on macOS (Clang) and Linux (GCC), and shall not rely on `__int128` without a portable fallback.	A1	T
NF-03	New code shall follow `docs/coding_convention.md` and carry Doxygen comments for public interfaces.	A1	R
NF-04	Line coverage of the new vector code shall be at least 90%, and branch coverage at least 75%.	A1/A2	T

7 Technical approach

This section fixes the architecture at the level needed for planning. The design document will cover data structures, algorithms and every legality rule in detail.

7.1 Module structure

Table 12: New modules and files.

New file or folder	Responsibility
`src/processor/vector/ vector_config.h, .cpp`	VLEN/ELEN parameters and the agnostic-fill policy
`src/processor/vector/vtype.h, .cpp`	Decoding, encoding and validation of `vtype`; VLMAX
`src/processor/vector/ vector_isa.h, .cpp`	The vector ISA table (D10): lookup by mnemonic, decoding, operand patterns, disassembly, expected-syntax text, pseudo-instruction names
`src/processor/vector/ vector_isa_table.inc`	Encodings generated from `riscv-opcodes`
`src/processor/vector/ vector_unit.h, .cpp`	`rvv::execute`: dispatch, legality checks, `vset{i}vl{i}`, atomic commit with undo records; exception types
`src/processor/vector/ vector_exec.h`	Execution state shared by the instruction groups: operand snapshot, working copy, element access, policies, legality helpers
`src/processor/vector/ vector_exec_integer.cpp`	Integer, fixed-point, mask, reduction and permutation instructions
`src/processor/vector/ vector_exec_memory.cpp`	Loads and stores
`src/processor/vector/ vector_exec_float.cpp`	Floating-point instructions
`src/assembler/parse_formats/ v_formats.cpp`	Generic operand-pattern parser for vector syntax
`tools/vector/` (Python)	Cross-check against `riscv-opcodes`; golden-encoding generation; Spike comparison; `instructions.json` generation
`test/vector/`, `examples/vector/`	Test suites and example programs

7.2 Execution flow

Figure 4 shows where the vector unit sits in the single-cycle processor.

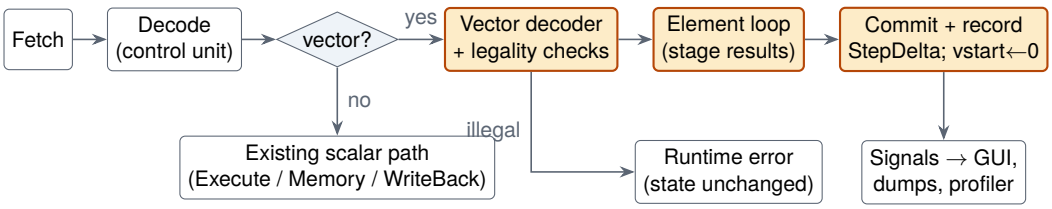


Figure 4: How RVSS executes an instruction after the change. Vector instructions are routed to the vector unit before the F/D checks, which closes the opcode-sharing hazard in F6.

7.3 The shared element loop

Every non-memory family runs through one loop, so masking, tail and `vstart` handling are written once:

```
checkLegality(insn, vtype)                 # throws before any state change (D5)
for i in 0 .. VLMAX-1:
    if i < vstart:                 continue          # prestart: never written
    if i >= vl:                    applyTail(i)      # vta policy (D3)
    else if masked and !v0.bit(i): applyInactive(i) # vma policy (D3)
    else:                          body(i)           # family-specific semantics
commitStagedWrites(); recordDelta(); vstart = 0
```

Reductions, mask-producing instructions, scalar moves, `vcompress` and memory instructions specialise this loop (for example, the destination EEW is 1 bit for compares, and a reduction writes only element 0). Element loops are $O(\text{VLMAX})$, which is at most 128 iterations for VLEN = 128, so they have no measurable effect on simulator speed.

7.4 Integration with existing subsystems

Assembler. New token types `VEC_REGISTER`, `VEC_MASK` (`v0.t`) and `VTYPE_FIELD`. Vtype fields are recognised only in operand position after `vsetvli`/`vsetivli` (AS-04). `ICUnit` gains `vm`, a vtype immediate and `nf`. Each table entry declares an operand pattern such as `"vd, vs2, vs1 [, vm]"`. The generic parser matches tokens against it, checks immediate ranges, fills the `ICUnit`, and builds the "expected syntax" text for errors from the same pattern.

Processor. The vector registers live in `RegisterFile`, next to the GPRs, FPRs and CSRs, so every processor model has them. The vector unit is a stateless function, `rvv::execute`, which RVSS calls from `Execute`. The 5-stage models get a decode-stage guard (D12), and B1 replaces it with real pipeline support.

Undo/redo. `StepDelta` gains `std::vector<VectorRegisterChange>`, holding old and new bytes for changed registers only. Vector CSR changes reuse the existing CSR records, and vector stores reuse `MemoryChange` (one record per contiguous byte run). At VLEN = 128, the worst-case undo footprint is about 1 KB per step.

Observability. `DumpRegisters` gains a `vector_registers` section, written as hex strings with element 0 in the least significant (rightmost) digits. The register view gets a third collapsible *Vector* section next to "General Purpose" and "Floating Point". The profiler gets `V-Type`.

Portability. The 64×64→128 multiply helper uses only 64-bit arithmetic, so it builds with any compiler (NF-02).

Appendix C maps every existing file that will change.

8 Implementation plan

8.1 Phases

Phases are ordered by dependency. Each phase ships with its own tests and ends with the full suite (old and new) passing; P6 adds system-level validation on top.

Table 13: Implementation phases.

Phase	Content	Exit criteria	Tier
P0	Tooling: LLVM (`llvm-mc`) and Spike; baseline build and test run; test skeletons.	Baseline results recorded.	—
P1	Foundations: vector register file, CSRs, `vtype` helpers, ISA table with `riscv-opcodes` cross-check, lexer and generic parser, encoder, disassembler, RVSS routing, atomic commit, undo/redo, dumps, pipeline guard, `csrr` pseudo-instructions.	`vset*` works end to end in GUI and CLI; `vtype`/VLMAX unit tests and undo property test pass.	A1
P2	Memory: unit-stride, strided, indexed, mask and whole-register loads and stores; EEW/EMUL legality.	`vvadd` and `memcpy` examples run; memory test matrix passes.	A1
P3	Integer arithmetic, compares, merge/move, widening/narrowing, multiply/divide/multiply-add, carry; masking and policy engine.	Integer test matrix passes for all 22 legal SEW/LMUL pairs.	A1
P4	Reductions, mask instructions, permutations; GUI vector section; profiler `V-Type`; A1 documentation.	Every A1 mnemonic has encoding and semantic tests; `docs/VECTOR.md` drafted.	A1
P5	Fixed-point; FP arithmetic, conversions, reductions and permutations; segment and fault-only-first; pseudo-instructions.	Every mnemonic in Appendix A has tests; FP edge-case suite passes.	A2
P6	System validation and documentation: Spike differential runs, coverage, negative tests, performance check, cross-platform build; design document and validation report; README, CLI and TESTING updates.	All quality gates in Section 9.6 met.	A1, A2
P7	Stretch goals, in order B1, B3, B6, B2, B4, B5, …	Each B item meets its own definition of done.	B

8.2 Ways of working

- **Branches:** `vector-extension` is the integration branch; the final PR goes to `main`.
- **Commits:** small and focused, following the repository's `type : summary` style (`feat`, `fix`, `test`, `docs`, `refac`, `build`).
- **Definition of done:** code, tests and documentation updated; full suite green; `clang-format` applied (UI `.ui` files excepted, as the coding convention requires); requirement IDs referenced in test names.

9 Validation and verification plan

9.1 Principles

1. **Independent oracles.** Expected values never come from the code under test. They come from Spike, LLVM, or a deliberately simple reference model written separately in the test code.
2. **Complete coverage of the ISA.** Every in-scope mnemonic has at least one encoding test and one semantic test. A meta-test walks the ISA table and fails if any entry has no tests.
3. **Positive and negative.** Correct results for legal use, and correct rejection of illegal use with state unchanged.
4. **Automated and repeatable.** Everything except the GUI checklist runs under CTest, using the existing presets.
5. **Traceable.** Test names carry requirement IDs, and the validation report contains a traceability matrix.

9.2 Test levels

Table 14: Test levels.

Level	Focus	What and how	Oracle / tool
L1	Units	`vtype` decode/encode, VLMAX/EMUL tables, legality predicates, element and group accessors, `vxrm` rounding helper, saturation, `rec7`/`rsqrt7` tables, 128-bit multiply helper.	Spec tables; hand-computed values
L2	Assembler and disassembler	For every mnemonic and operand form: assemble and compare the word with the golden encoding; boundary immediates; masked and unmasked forms; disassembler round trip; error cases (bad register, out-of-range immediate, missing operand, `v0.t` where not allowed); labels named like vtype fields.	`llvm-mc` [7] encodings; `riscv-opcodes` [4]
L3	Instruction semantics	Parameterised GoogleTests per family over the configuration matrix (Table 15), with edge-value operands: 0, ± 1, signed min/max, unsigned max; for FP, ± 0, $\pm\infty$, quiet and signalling NaN, subnormals and rounding ties.	Independent C++ reference model; Spike golden vectors for spot checks
L4	Legality	One test per rule in A1.9: the instruction raises the right error and leaves state (registers, vector registers, CSRs, memory) bit-identical.	Spec rules
L5	Programs	Example programs (Section 9.5) run on RVSS through `assemble()` and `DebugRun()`. Results are checked against expected memory and registers, and against scalar versions of the same algorithm also run in Kites.	Scalar Kites run; Spike
L6	Differential vs Spike	The same programs on Spike (`--isa=rv64gcv --varch=vlen:128,elen:64`) and on Kites; compare GPRs, FPRs, vector registers, vector CSRs and a memory window. A covers the example and curated programs; B6 adds random programs at scale.	Spike [6]

Table 14: Test levels (continued).

Level	Focus	What and how	Oracle / tool
L7	System behaviour	Undo/redo property test: run N steps, undo N, compare with the initial state bit-exactly; redo N and compare with the final state. Also reset, breakpoints inside vector loops, the dump JSON schema, profiler counts, and the pipeline guard on all four RV5S models.	Recorded state
L8	Regression and platforms	Full existing suite on every merge; builds on macOS (Clang) and Linux (GCC); Windows via the release workflow.	Existing tests
L9	Coverage	gcovr through the existing `coverage` target, with thresholds from NF-04.	gcovr
L10	GUI (manual)	Checklist: vector section updates on step, run, undo, redo and reset; highlighting; syntax colouring and autocompletion; error markers on the right line.	Checklist in the validation report

9.3 Configuration test matrix

Table 15: Dimensions of the semantic test matrix. Every family is swept over all 22 legal SEW/LMUL pairs with representative values of the other dimensions; the remaining dimensions are covered pairwise.

Dimension	Values
SEW × LMUL	All legal pairs: $7 + 6 + 5 + 4 = 22$ for ELEN = 64 (Table 5); illegal pairs are covered by the `vill` tests
`vl`	0, 1, $\lfloor \text{VLMAX}/2 \rfloor + 1$, VLMAX$-1$, VLMAX; AVL $>$ VLMAX and AVL $\geq 2\cdot$VLMAX
Mask	Unmasked; masked with all ones, all zeros, alternating and random patterns
Policy	`tu,mu`; `ta,ma`; mixed; with the all-ones agnostic fill when B4 exists
`vstart`	0; 1; `vl`-1; `vstart` $\geq$ `vl` (no body elements)
Register overlap	`vd` = `vs1`, `vd` = `vs2`, `vs1` = `vs2` where the spec allows it; forbidden overlaps in L4
Memory	EEW 8–64 with every legal EMUL; strides 0, ±EEW, large and non-multiple; index patterns identity, reversed, duplicate and random; misaligned base addresses
FP	`frm` = RNE, RTZ, RDN, RUP, RMM, and dynamic; flag accrual over sequences
Fixed point	`vxrm` = rnu, rne, rdn, rod; saturating and non-saturating inputs

9.4 Reference oracles and golden sources

Spike (riscv-isa-sim) [6]
The de-facto RISC-V reference simulator, with complete RVV 1.0 support and configurable VLEN/ELEN. It is the main oracle for L3 spot checks and for L5/L6.

LLVM MC [7] and GNU binutils
Golden encodings and disassembly text for L2. Generated golden files are committed, so tests do not need LLVM at run time.

riscv-opcodes [4]
Machine-readable match/mask values for every vector instruction. They are used to cross-check the ISA table automatically.

Spec Appendix B [2]
Official vector code examples (memcpy, conditional execution, SAXPY, SGEMM, division and square-root approximations, `strlen`, `strcmp`, `strcpy`, `strncpy`) that become L5 programs.

riscv-vector-tests [8]
Spike-generated per-instruction tests for VLEN 64 to 4096. Kites cannot load ELF files, so test vectors are extracted rather than run directly. These tests do not cover tail/mask-agnostic behaviour, fault-only-first, `vstart` or overlap, which is why L3/L4 cover those explicitly.

RVVTS [9]
Coverage-driven RVV test sets reported to reach over 94% functional coverage. Used as a benchmark for test-coverage goals and, optionally, as an extra test source.

Independent reference model
A small, loop-based C++ model in the test tree, written from the spec text without looking at the implementation.

Known legitimate differences from Spike

The spec allows implementation-defined choices, and the differential tests must not flag them as bugs: (1) values of agnostic elements (Kites leaves them undisturbed, D3); (2) the summation order of `vfredusum`/`vfwredusum`, which can change the last bits (compared using exactly representable inputs, or with an ordered-sum oracle); (3) `vl` for VLMAX $<$ AVL $<$ 2·VLMAX (Kites uses D2, the same as Spike's default). Every difference found is either fixed or recorded with its justification in the validation report.

9.5 Example programs

The programs in `examples/vector/` are run as integration tests (L5, L6):

- E1 `vvadd_i32.s`: strip-mined vector add;
- E2 `memcpy.s`: byte copy with `vle8.v`/`vse8.v` (spec Appendix B);
- E3 `dot_product.s`: `vmul` with `vredsum`;
- E4 `masked_abs.s`: conditional update with `vmslt` and a masked `vrsub`;
- E5 `gather_scatter.s`: table lookup and histogram with indexed accesses;
- E6 `prefix_sum.s`: log-step scan with `vslideup`;
- E7 `matmul_i32.s`: integer matrix multiply, checked against the existing scalar program (`cache_matrix_mul_3x3.s`);
- E8 `saxpy_f32.s`: SAXPY with `vfmacc.vf` (A2, spec Appendix B);
- E9 `strlen.s`: `vle8ff.v` with `vmseq` and `vfirst.m` (A2, spec Appendix B);
- E10 `rgb_to_grey.s`: segment loads `vlseg3e8.v` with widening multiply and narrowing (A2);
- E11 `fir_q15.s`: fixed-point FIR filter with `vsmul` and `vxrm` (A2).

9.6 Quality gates

Table 16: Quality gates for completion of A1 and A2.

QG1	100% of in-scope mnemonics have encoding and semantic tests (enforced by the meta-test).
QG2	Zero failing tests, and zero known deviations from the spec without a documented reason.
QG3	At least 90% line coverage and 75% branch coverage on `src/processor/vector/` and `v_formats.cpp`.
QG4	All example programs match Spike bit-exactly (subject to the known differences above).
QG5	The existing suite passes on macOS and Linux, and scalar run time is within ±5% of the baseline.
QG6	Undo/redo property tests pass for every example program.
QG7	The GUI checklist is fully ticked, with screenshots in the validation report.

9.7 Traceability and defect tracking

Each test name starts with the ID of the requirement it verifies. For example, the test `VM02_negative_stride` in suite `VecMem` checks requirement VM-02. The validation report has a requirement-to-test-to-result matrix generated from the CTest output. Defects are GitHub issues on the fork, labelled `vector`, each with a minimal reproducing `.s` file that becomes a regression test when the defect is fixed.

10 Documentation plan

Table 17: Documentation deliverables.

Document	Format	Audience	Content
Scope document (this document)	LaTeX → PDF	Reviewers	Scope, requirements, approach, validation and documentation plans
Design document	LaTeX → PDF	Reviewers, developers	Architecture, data structures, ISA table, algorithms, legality rules, integration details
Validation report	LaTeX → PDF	Reviewers	Results per test level, coverage, Spike comparison, traceability matrix, known limitations
`docs/VECTOR.md`	Markdown	Users	Supported instructions, syntax, configuration, examples, limitations
`README.md`, `docs/CLI.md`, `docs/TESTING.md`	Markdown	Users, developers	Feature list (including the F15 fix), vector dump format, running the vector tests
`docs/vector_extension/README.md`	Markdown	Developers	Index of these documents; how to build the PDFs
Doxygen comments	In code	Developers	All public vector interfaces
Example headers	Comments	Users	Purpose, inputs and expected results of each `.s` file

The LaTeX sources live in `docs/vector_extension/`, share the style file `kitesdoc.sty`, and build with `tectonic` (or `make`). The PDFs are committed next to their sources.

11 Risks and mitigations

Table 18: Risks and mitigations.

ID	Risk	L	I	Mitigation
R1	With 627 mnemonics, the encoder, decoder and executor drift apart.	M	H	Single ISA table (D10); `riscv-opcodes` cross-check; table-completeness meta-test.
R2	Subtle spec rules (overlap, EMUL, policies, `vstart`) are implemented wrongly.	H	H	One legality module with a test per rule; shared element loop; Spike differential tests; review against the spec text.
R3	Sharing opcodes with scalar FP loads/stores breaks F/D.	M	H	Route on `width` before the F/D checks; run the existing FP suites on every merge (SI-03).
R4	FP corner cases (NaNs, flags, rounding, `rec7` tables) differ from Spike.	M	M	Own spec-conformant element routines (D9); dedicated FP edge-case suite.
R5	Assembler ambiguity (`v0.t` lexing, labels named like vtype fields).	M	M	Context-sensitive lexing; tests AS-04.
R6	Portability (`__int128`; Windows MinGW release builds).	L	M	Portable 128-bit multiply helper; build on macOS and Linux; Windows through the release workflow.
R7	GUI slows down with many register updates or a large VLEN.	L	M	One batched update per step; refresh only changed registers.
R8	Spike or LLVM is hard to install on the development machine.	M	M	Homebrew packages; committed golden files; the independent C++ oracle needs no external tools.
R9	Pipeline integration (B1) is more invasive than expected.	M	M	Kept out of the compulsory scope; the D12 guard keeps the pipelines safe in the meantime.
R10	Existing scalar issues (F8, F15) interfere with vector work.	M	L	Fix them in small, separate, tested commits, and report them upstream.
R11	Upstream Kites changes conflict with the branch.	L	M	Keep the hooks small (D13); rebase regularly.

L = likelihood, I = impact; H = high, M = medium, L = low.

12 Open questions

Implementation proceeds with the default shown for each question until it is answered.

- Q1 **Pipelined vector execution compulsory?** Default: no, stretch goal B1. If compulsory, B1 moves to A and A2.4–A2.5 move to Tier B.
- Q2 **Default VLEN.** Default: 128, the minimum for **V** (D1).
- Q3 **Agnostic policy.** Default: undisturbed, with all-ones as an option (D3).
- Q4 **FP vector instructions compulsory?** Default: yes; **V** requires them and Kites already implements F and D.
- Q5 **Upstream contribution.** Default: the work targets the fork; a PR to `hghit-harshit/Kites` is

optional.

References

[1] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, release 20260120, Chapter 30: "V" Standard Extension for Vector Operations, version 1.0. https://docs.riscv.org/reference/isa/v20260120/unpriv/v-st-ext.html

[2] RISC-V International. *Unprivileged ISA manual*, Appendix B: Vector Assembly Code Examples. https://docs.riscv.org/reference/isa/v20260120/unpriv/vector-examples.html

[3] RISC-V International. *riscv-v-spec*, the original repository of the RVV 1.0 specification. https://github.com/riscv/riscv-v-spec

[4] RISC-V International. *riscv-opcodes* (vector encodings in `extensions/rv_v`). https://github.com/riscv/riscv-opcodes

[5] RISC-V International. *RISC-V Assembly Programmer's Manual*. https://github.com/riscv-non-isa/riscv-asm-manual

[6] RISC-V Software. *Spike RISC-V ISA Simulator*. https://github.com/riscv-software-src/riscv-isa-sim

[7] LLVM Project. *User Guide for RISC-V Target* and `llvm-mc`. https://llvm.org/docs/RISCVUsage.html

[8] CHIPS Alliance. *riscv-vector-tests*: a generator for RISC-V vector unit tests. https://github.com/chipsalliance/riscv-vector-tests

[9] Institute for Complex Systems, JKU Linz. *RVVTS: RISC-V Vector Test Suite*. https://github.com/ics-jku/RVVTS

[10] J. L. Hennessy and D. A. Patterson. *Computer Architecture: A Quantitative Approach*, 6th ed., Chapter 4: Data-Level Parallelism in Vector, SIMD, and GPU Architectures. Morgan Kaufmann, 2017.

[11] M. Perotti et al. "A 'New Ara' for Vector Computing: An Open Source Highly Efficient RISC-V V 1.0 Vector Processor Design." *ASAP 2022*. arXiv:2210.08882.

[12] Kites RISC-V simulator. https://github.com/hghit-harshit/Kites

A Instruction inventory

Mnemonics are grouped by family. Braces expand to alternatives, for example `vadd.{vv,vx,vi}` means three mnemonics. EEW $\in \{8, 16, 32, 64\}$ and NF $\in \{2, \ldots, 8\}$. The counts were produced by a script that enumerates the specification's instruction listings.

Table 19: RVV 1.0 instruction inventory by family, with tier.

Family	Mnemonics	#	Tier
Configuration and memory			
Configuration	`vsetvli`, `vsetivli`, `vsetvl`	3	A1
Unit-stride	`vle<EEW>.v`, `vse<EEW>.v`	8	A1
Mask load/store	`vlm.v`, `vsm.v`	2	A1
Strided	`vlse<EEW>.v`, `vsse<EEW>.v`	8	A1
Indexed	`vl{u,o}xei<EEW>.v`, `vs{u,o}xei<EEW>.v`	16	A1
Whole register	`vl{1,2,4,8}re<EEW>.v`, `vs{1,2,4,8}r.v`	20	A1
Fault-only-first	`vle<EEW>ff.v`	4	A2
Segment	`vlseg<NF>e<EEW>{,ff}.v`, `vsseg`, `vlsseg`, `vssseg`, `vl{u,o}xseg<NF>ei<EEW>.v`, `vs{u,o}xseg<NF>ei<EEW>.v`	252	A2
Integer arithmetic			
Add / subtract	`vadd.{vv,vx,vi}`, `vsub.{vv,vx}`, `vrsub.{vx,vi}`	7	A1
Widening add/sub	`vw{add,sub}{,u}.{vv,vx,wv,wx}`	16	A1
Extension	`v{z,s}ext.vf{2,4,8}`	6	A1
Carry / borrow	`vadc.{vvm,vxm,vim}`, `vmadc.{vvm,vxm,vim,vv,vx,vi}`, `vsbc.{vvm,vxm}`, `vmsbc.{vvm,vxm,vv,vx}`	15	A1
Bitwise	`v{and,or,xor}.{vv,vx,vi}`	9	A1
Shifts	`v{sll,srl,sra}.{vv,vx,vi}`	9	A1
Narrowing shifts	`vn{srl,sra}.{wv,wx,wi}`	6	A1
Compare	`vms{eq,ne,leu,le}.{vv,vx,vi}`, `vms{ltu,lt}.{vv,vx}`, `vms{gtu,gt}.{vx,vi}`	20	A1
Min / max	`v{min,max}{,u}.{vv,vx}`	8	A1
Multiply	`vmul`, `vmulh`, `vmulhu`, `vmulhsu .{vv,vx}`	8	A1
Divide / remainder	`v{div,rem}{,u}.{vv,vx}`	8	A1
Widening multiply	`vwmul{,u,su}.{vv,vx}`	6	A1
Multiply-add	`v{macc,nmsac,madd,nmsub}.{vv,vx}`	8	A1
Widening multiply-add	`vwmacc{,u,su}.{vv,vx}`, `vwmaccus.vx`	7	A1
Merge / move	`vmerge.{vvm,vxm,vim}`, `vmv.v.{v,x,i}`	6	A1
Fixed-point			
Saturating add/sub	`vsadd{,u}.{vv,vx,vi}`, `vssub{,u}.{vv,vx}`	10	A2
Averaging add/sub	`va{add,sub}{,u}.{vv,vx}`	8	A2
Fractional multiply	`vsmul.{vv,vx}`	2	A2
Scaling shifts	`vss{rl,ra}.{vv,vx,vi}`	6	A2
Narrowing clips	`vnclip{,u}.{wv,wx,wi}`	6	A2
Floating-point			
Add / subtract	`vf{add,sub}.{vv,vf}`, `vfrsub.vf`	5	A2
Widening add/sub	`vfw{add,sub}.{vv,vf,wv,wf}`	8	A2
Multiply / divide	`vf{mul,div}.{vv,vf}`, `vfrdiv.vf`	5	A2

Table 19: RVV 1.0 instruction inventory by family, with tier (continued).

Family	Mnemonics	#	Tier
Widening multiply	`vfwmul.{vv,vf}`	2	A2
Fused multiply-add	`vf{,n}m{acc,sac,add,sub}.{vv,vf}`	16	A2
Widening FMA	`vfw{,n}m{acc,sac}.{vv,vf}`	8	A2
Unary	`vfsqrt.v`, `vfrsqrt7.v`, `vfrec7.v`, `vfclass.v`	4	A2
Min / max	`vf{min,max}.{vv,vf}`	4	A2
Sign injection	`vfsgnj{,n,x}.{vv,vf}`	6	A2
Compare	`vmf{eq,ne,lt,le}.{vv,vf}`, `vmf{gt,ge}.vf`	10	A2
Merge / move	`vfmerge.vfm`, `vfmv.v.f`	2	A2
Conversions	`vfcvt.*` (6), `vfwcvt.*` (7), `vfncvt.*` (8), including `rtz` and `rod`	21	A2
Reductions, masks and permutations			
Integer reductions	`vred{sum,and,or,xor,min,minu,max,maxu}.vs`, `vwredsum{,u}.vs`	10	A1
FP reductions	`vfred{osum,usum,min,max}.vs`, `vfwred{osum,usum}.vs`	6	A2
Mask instructions	`vm{and,nand,andn,xor,or,nor,orn,xnor}.mm`, `vcpop.m`, `vfirst.m`, `vms{bf,if,of}.m`, `viota.m`, `vid.v`	15	A1
Integer permutations	`vmv.x.s`, `vmv.s.x`, `vslide{up,down}.{vx,vi}`, `vslide1{up,down}.vx`, `vrgather.{vv,vx,vi}`, `vrgatherei16.vv`, `vcompress.vm`, `vmv{1,2,4,8}r.v`	17	A1
FP permutations	`vfmv.f.s`, `vfmv.s.f`, `vfslide1{up,down}.vf`	4	A2
Total: A1 238 + A2 389		**627**	

B Assembly syntax forms

Square brackets mark the optional mask operand. The generic parser handles each row as one operand pattern.

Table 20: Assembly operand patterns.

Pattern	Example	Used by
`rd, rs1, vtypei`	`vsetvli t0, a0, e32, m2, ta, ma`	`vsetvli`
`rd, uimm, vtypei`	`vsetivli t0, 8, e8, mf2, tu, mu`	`vsetivli`
`rd, rs1, rs2`	`vsetvl t0, a0, a1`	`vsetvl`
`vd, (rs1) [, v0.t]`	`vle32.v v4, (a1), v0.t`	unit-stride, ff, segment, mask, whole register
`vd, (rs1), rs2 [, v0.t]`	`vlse64.v v8, (a1), t1`	strided
`vd, (rs1), vs2 [, v0.t]`	`vluxei32.v v8, (a1), v4`	indexed
`vd, vs2, vs1 [, v0.t]`	`vadd.vv v1, v2, v3, v0.t`	`.vv`, `.wv`, `.vs`, `.mm`
`vd, vs2, rs1 [, v0.t]`	`vsub.vx v1, v2, a0`	`.vx`, `.wx`, slides
`vd, vs2, imm [, v0.t]`	`vsll.vi v1, v2, 3`	`.vi`, `.wi`
`vd, vs2, fs1 [, v0.t]`	`vfmul.vf v1, v2, fa0`	`.vf`, `.wf`
`vd, vs1, vs2 [, v0.t]`	`vmacc.vv v1, v2, v3`	integer and FP multiply-add (note the operand order)

Table 20: Assembly operand patterns (continued).

Pattern	Example	Used by
`vd, rs1 / fs1, vs2 [, v0.t]`	`vfmacc.vf v1, fa0, v2`	multiply-add, scalar forms
`vd, vs2, vs1 / rs1 / imm, v0`	`vmerge.vvm v1, v2, v3, v0`	carry/borrow, merge (`v0` is an operand, not a mask)
`vd, vs1 / rs1 / imm / fs1`	`vmv.v.x v1, a0`	`vmv.v.*`, `vfmv.v.f`
`rd, vs2 / fd, vs2`	`vmv.x.s a0, v1`	`vmv.x.s`, `vfmv.f.s`
`vd, rs1 / vd, fs1`	`vmv.s.x v1, a0`	`vmv.s.x`, `vfmv.s.f`
`vd, vs2 [, v0.t]`	`vzext.vf2 v2, v1`	unary, conversions, `vmsbf.m`, `viota.m`
`rd, vs2 [, v0.t]`	`vcpop.m a0, v0`	`vcpop.m`, `vfirst.m`
`vd [, v0.t]`	`vid.v v1`	`vid.v`
`vd, vs2, vs1`	`vcompress.vm v1, v2, v0`	`vcompress.vm`
`vd, vs2`	`vmv2r.v v2, v4`	whole-register move

C Impact on existing Kites files

Table 21: Existing files affected, by tier.

File(s)	Change	Tier
`src/processor/registers.h`, `registers.cpp`	Vector register storage; vector CSR names and addresses; `vcsr` view; `v0`–`v31` names; change signals	A1
`src/common/register_names.h`	Vector register names	A1
`src/common/instructions.h`, `instructions.cpp`	`isVectorInstruction()`; hooks into validation, type lookup, expected-syntax text and `disassemble()`	A1
`src/common/instruction_types.h`	`V_TYPE` category	A1
`src/assembler/tokens.h`, `lexer.cpp`	`VEC_REGISTER`, `VEC_MASK`, `VTYPE_FIELD`; context-sensitive lexing	A1
`src/assembler/parser.h`, `parser.cpp`	Dispatch to the generic vector parser	A1
`src/assembler/code_generator.h`, `code_generator.cpp`	New `ICUnit` fields; vector encoder	A1
`src/assembler/parse_formats/pseudo_formats.cpp`	`csrr`-style pseudo-instructions (A1); vector pseudo-instructions (A2)	A1, A2
`src/processor/processor_base.h`	`StepDelta` vector register changes	A1
`src/processor/rvss/rvss_processor.cpp`, `rvss_control_unit.cpp`	Routing, commit, undo/redo, reset	A1
`src/processor/rv5s/*`	Decode guard (A1); full integration including `rv5s_hdu.cpp` and `pipeline_registers.h` (B1)	A1, B1
`src/utils/utils.cpp`	`vector_registers` section in the register dump	A1
`src/ui/register_table/*`	*Vector* section (A1); element-wise viewer (B3)	A1, B3
`resources/instructions/instructions.json`	`VInstruction` list (generated)	A1

Table 21: Existing files affected, by tier (continued).

File(s)	Change	Tier
`src/language_service/completion_provider.cpp`, `src/ui/common/syntax_highlighter.cpp`	Check and extend for vector registers and mnemonics	A1
`src/profiler/*`	`V-Type` (A1); vector metrics (B5)	A1, B5
`src/config/config.h`, `src/ui/dialogs/settings_dialog.*`	VLEN and agnostic-policy settings	B4
`cli/main.cpp`, `cli/repl.cpp`	Documentation (A1); flags and commands (B4, B9)	A1, B
`src/ui/processor_tab/*`	Vector unit diagram	B7

D Glossary

Term	Meaning
AVL	Application vector length: the number of elements software still wants to process.
VLEN	Width of each vector register in bits (128 by default in Kites).
ELEN	Largest supported element width in bits (64).
SEW / EEW	Selected element width from `vtype`; effective element width of a particular operand.
LMUL / EMUL	Register-group multiplier from `vtype`; effective multiplier of a particular operand.
VLMAX	Maximum number of elements an instruction can process: LMUL$\times$VLEN/SEW.
vl	Number of elements the current instruction processes ($\leq$ VLMAX).
Prestart / body / tail	Elements below `vstart`; from `vstart` to `vl`-1; from `vl` to VLMAX-1.
Active / inactive	Body elements whose mask bit is 1 / 0.
Undisturbed / agnostic	Policy for inactive and tail elements: keep the old value / old value or all ones.
Strip-mining	Processing a long array in chunks of `vl` elements per loop iteration.
Segment access	A load or store that moves records of NFIELDS fields into NFIELDS separate register groups.
Fault-only-first	A load that traps only on element 0 and otherwise shortens `vl`.
Chime, lane, chaining	Timing concepts (B2): the time to issue one vector instruction; a parallel datapath slice; starting a dependent instruction before its producer finishes.
Oracle	An independent source of expected results used by a test.